

Exerciții — operații pe tabel, pas cu pas

Zece butoane, o operație fiecare.

Fișa asta nu construiește o aplicație. Construiește **zece butoane**, fiecare cu o singură operație pe tabel. Le faci pe rând, în ordine, pe aceeași formă.

Nu e CRUD. CRUD-ul vine după, și o să înțelegi de ce abia când termini fișa asta.

Datele de pornire

Ai nevoie de ceva în tabel ca să te joci cu el. Două clase, în fișierele lor:

```
class Echipa
{
    public string nume;
    public string oras;
}

class Jucator
{
    public string nume;
    public int varsta;
    public string pozitie;
    public int goluri;
    public Echipa echipa;
}
```

`Jucator` ține un **obiect** `Echipa`, nu un text. La afișare scrii prin el: `j.echipa.nume`.

Forma se cheamă `JucatoriForm` și are: un `DataGridView` numit `dgvJucatori`, o etichetă `lblStare`, și butoanele pe care le adaugi exercițiu cu exercițiu.

Incarca jucatorii

Incarcati 8 jucatori

Jucator	Varsta	Pozitie	Echipa	Orasul echipei	Goluri	Goluri pe meci
Popescu Andrei	24	atacant	CFR Cluj	Cluj-Napoca	14	0.50
Ionescu Vlad	29	mijlocas	CFR Cluj	Cluj-Napoca	6	0.20
Marin Radu	21	atacant	FCSB	Bucuresti	9	0.41
Dumitru Alex	33	portar	FCSB	Bucuresti	0	0.00
Stan Cristian	26	fundas	Rapid	Bucuresti	2	0.07
Neagu Mihai	19	mijlocas	Rapid	Bucuresti	4	0.27
Barbu Daniel	31	atacant	CFR Cluj	Cluj-Napoca	17	0.59
Vlad Sergiu	23	fundas	FCSB	Bucuresti	1	0.04

Tabelul de pornire — asta trebuie să-ți iasă după exercițiul 1

Partea 1 — tabelul se umple și se golește

1. Încarcă

1. Scrie `CreateColumns()` cu șase coloane — `nume`, `varsta`, `pozitie`, `echipa`, `oras`, `goluri` — cu antete scrise omenește. Cheam-o din constructor, **o singură dată**.
2. Scrie `IncarcaJucatori()` care întoarce `List<Jucator>` cu opt jucători de la trei echipe. Echipele le construiești o dată, la începutul metodei.
3. Ține lista într-un **câmp al formei**, nu într-o variabilă locală: `private List<Jucator> jucatori = new List<Jucator>();`
4. Butonul `btnIncarca` umple tabelul cu bucla din tipar.

Gata când: apeși de trei ori la rând și sunt tot opt rânduri.

2. Golește tabelul

Buton `btnGoleste`, cu **o singură linie**: `dgvJucatori.Rows.Clear();`

Apasă apoi `btnIncarca`. Cei opt jucători se întorc.

Scrie într-o etichetă răspunsul la întrebarea: dacă ai golit tabelul, de unde au reapărut?

Gata când: golești, încarci, golești, încarci — și de fiecare dată sunt tot opt.

3. Numără

Buton `btnNumara`. În `lblStare` scrie câte **rânduri** și câte **coloane** are tabelul.

Ia numerele din tabel: `dgvJucatori.Rows.Count` și `dgvJucatori.Columns.Count`. Nu din listă, nu scrise de mână.

Gata când: golești tabelul, apeși **Numara** , și scrie 0 randuri, 6 coloane .

Partea 2 — rânduri: adaugi, ștergi, citești

4. Adaugă un rând direct în tabel

Buton `btnAdaugaRand` și o casetă `txtNume`.

La apăsare, adaugi în tabel un rând nou cu numele din casetă și restul valorilor inventate:

```
dgvJucatori.Rows.Add(txtNume.Text, 20, "mijlocas", "FCSB",  
                    "Bucuresti", 0);
```

Nu atingi lista `jucatori`.

Acum partea interesantă: după ce ai adăugat rândul, apasă `btnIncarca`.

Gata când: rândul adăugat **dispare** după `Incarca`. Scrie pe formă de ce.

5. Șterge rândul selectat

Buton `btnStergeRand`.

1. Verifică întâi că există un rând selectat: `dgvJucatori.CurrentRow` nu e `null`.
2. Dacă nu e, mesaj și te oprești.
3. Dacă e: `dgvJucatori.Rows.RemoveAt(dgvJucatori.CurrentRow.Index);`

Gata când: apeși `Sterge înainte` să fi încărcat ceva și aplicația rămâne deschisă; iar un rând șters re apare după `Incarca`.

6. Citește o celulă

Fără buton. Folosește evenimentul `SelectionChanged` al tabelului.

Când selectezi un rând, în `lblStare` apare numele și echipa jucătorului de pe rândul ăla, citite **din celule**:

```
string nume = dgvJucatori.CurrentRow.Cells["nume"].Value.ToString();
```

Verifică `CurrentRow` înainte — evenimentul se declanșează și când tabelul se golește.

Gata când: te muți prin tabel cu **săgețile de pe tastatură** și eticheta se schimbă și așa, nu doar la click.

Partea 3 — celule și parcurgere

7. Modifică o celulă

Buton `btnGolMarcat` . Adaugă **un gol** jucătorului de pe rândul selectat, scriind direct în celulă.

1. Citește valoarea curentă din celula `goluri` .
2. Transform-o în `int` .
3. Scrie înapoi valoarea plus unu.

🔧 `Cells[...].Value` este de tip `object` , nu `int` . Nu poți face `+ 1` direct pe el.

Gata când: apeși de trei ori și numărul crește cu 3. Apoi apeși `Incarca` — și scrie pe formă ce s-a întâmplat cu golurile adăugate.

8. Total pe o coloană

Buton `btnTotal` . În `lblStare` scrie câte goluri sunt în total.

Suma o calculezi **parcurgând tabelul**, nu lista:

1. `for` de la `0` la `dgvJucatori.Rows.Count` .
2. La fiecare pas, iei valoarea din celula `goluri` a rândului `i` .
3. O transformi în `int` și o aduni.

Gata când: ștergi un rând cu `btnStergeRand` , apeși `Total` , și suma e mai mică cu golurile jucătorului șters.

Partea 4 — selecție și culoare

9. Caută și selectează

Casetă `txtCauta` și buton `btnCauta`.

Parcurgi rândurile și **selectezi** primul rând al cărui nume conține textul tastat:

```
dgvJucatori.CurrentCell = dgvJucatori.Rows[i].Cells[0];
```

Dacă nu găsește niciunul, `lblStare` anunță asta. Tabelul rămâne cum era — nu ștergi rânduri, doar muți selecția.

Gata când: cauți o bucată de nume, rândul potrivit se evidențiază singur, iar eticheta de la exercițiul 6 arată jucătorul găsit.

10. Colorează rândurile

Buton `btnColoreaza`. Rândurile jucătorilor cu cel puțin 10 goluri devin verzi, restul rămân albe.

```
dgvJucatori.Rows[i].DefaultCellStyle.BackColor = Color.LightGreen;
```

Parcurgi toate rândurile. Pe cele care nu trec condiția le pui pe `Color.White` — altfel, la a doua apăsare, culorile rămân de la prima.

Gata când: apeși `Coloreaza`, apoi adaugi un gol cu `btnGolMarcat` unui jucător care avea 9, apeși iar `Coloreaza` — și rândul lui devine verde.

Ce ai observat, de fapt

Uită-te înapoi la exercițiile 4, 5 și 7. În toate trei ai modificat **tabelul**: ai adăugat un rând, ai șters un rând, ai schimbat un număr. Și de fiecare dată, un `Incarca` a șters tot ce făcuseși.

Tabelul e o **vitrină**. Ce vezi în el sunt copii ale datelor, nu datele. Adevărul stă în `List<Jucator>`, iar tabelul se reface din ea de fiecare dată.

De asta o aplicație adevărată nu modifică tabelul: modifică **lista**, apoi reafișează. Aia e partea care se cheamă CRUD, și o facem data viitoare — acum știi deja de ce e nevoie de ea.

Înainte să-mi zici că e gata

- apeși fiecare buton **de trei ori la rând** — a treia apăsare face la fel ca prima?
- apeși butoanele care lucrează cu selecția **înainte** să încarci ceva?
- lași caseta goală și apeși **Cauta** ?
- `CreateColumns()` se cheamă o singură dată, din constructor?
- ai vreun handler numit `button1_Click` ?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro